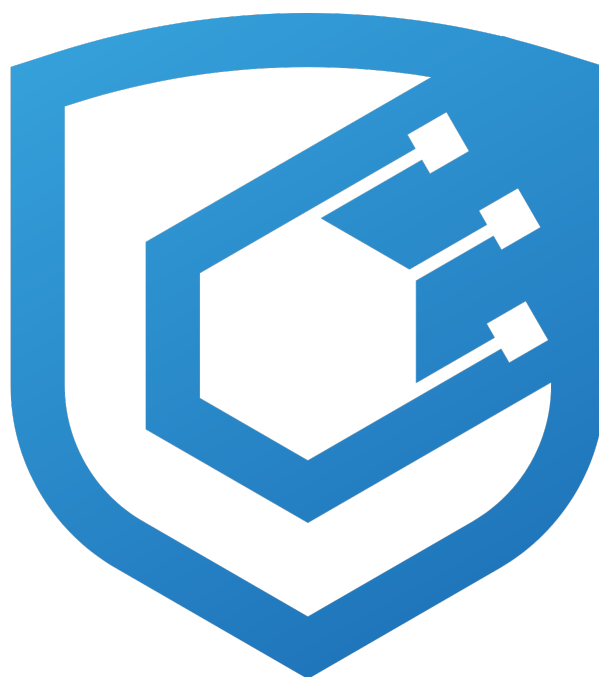




Securitize DSToken Compliance Diff Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Kage](#)

[SBSecurity](#) ([Blckhv](#), [Slavcheww](#))

June 4, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	2
7	Findings	4
7.1	Medium Risk	4
7.1.1	ComplianceServicePermissionless::_lockedAt and ComplianceServicePermissionless::_cleanupIssuances perform unguarded timestamp + lockPeriod addition, allowing a transfer agent to freeze transfers and issuances	4
7.1.2	TokenLibrary::issueTokensCustom routes every manual lock under permissionless into a single shared investorsLocks[""] bucket, exhausting the global 30-lock cap and DoS-ing all future issuances with manual locks	7
7.1.3	ComplianceServicePermissionless::recordIssuance writes a lockup record even when the lock period is zero, causing tokens minted with no lockup to be retroactively locked when a lockup is later enabled	10
7.2	Low Risk	13
7.2.1	ComplianceServicePermissionless::preIssuanceCheck omits the pause check, allowing issuance while the token is paused contrary to FR-4	13
7.2.2	WalletManager::setSpecialWallet guard is inert under the stub registry, letting an Issuer label any token-holding wallet as a platform wallet and bypass an active lockup	14

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Securitize DSToken is a permissioned ERC-20 security-token framework whose transfers are gated by a pluggable compliance service. PR-87 adds a permissionless variant (ComplianceServicePermissionless) paired with a no-op StubRegistryService, allowing tokens to be held and transferred by any wallet with no on-chain investor identity. On-chain enforcement is reduced to a blacklist, pause, balance, and a configurable post-issuance lockup. KYC and investor identity are handled entirely off-chain.

5 Audit Scope

The changes specific to PR-87 were in scope for this audit. Changes in following files were in scope.

```
ComplianceService.sol
ComplianceServicePermissionless.sol
ComplianceServiceRegulated.sol
ComplianceServicePermissionlessDataStore.sol
StubRegistryService.sol
```

6 Executive Summary

Over the course of 2 days, the Cyfrin team conducted an audit on the [DSToken Compliance Diff](#) code provided by [Securitize](#). The findings consist of 3 Medium & 2 Low severity issues.

The review covered the [PR #87 diff](#) introducing the Securitize DSToken permissionless compliance pathway.

The main findings centered on the interaction between permissionless wallet-based compliance and legacy investor-based assumptions. Key themes included lockup accounting edge cases, stale or unintended lock records, registry-stub behavior that changed prior invariants, and compliance-rule mismatches around pause handling and platform-wallet exemptions.

All reported issues were fixed by the Securitize team and verified by Cyfrin as part of the mitigation review.

Summary

Project Name	DSToken Compliance Diff
Repository	dstoken
Commit	819536161174...
Fix Commit	0f89bd19f33d...
Audit Timeline	Apr 23rd - Apr 25th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	3
Low Risk	2
Informational	0
Gas Optimizations	0
Total Issues	5

Summary of Findings

[M-1] <code>ComplianceServicePermissionless::_lockedAt</code> and <code>ComplianceServicePermissionless::_cleanupIssuances</code> perform unguarded <code>timestamp + lockPeriod</code> addition, allowing a transfer agent to freeze transfers and issuances	Resolved
[M-2] <code>TokenLibrary::issueTokensCustom</code> routes every manual lock under <code>permissionless</code> into a single shared <code>investorsLocks[""]</code> bucket, exhausting the global 30-lock cap and DoS-ing all future issuances with manual locks	Resolved
[M-3] <code>ComplianceServicePermissionless::recordIssuance</code> writes a lockup record even when the lock period is zero, causing tokens minted with no lockup to be retroactively locked when a lockup is later enabled	Resolved
[L-1] <code>ComplianceServicePermissionless::preIssuanceCheck</code> omits the pause check, allowing issuance while the token is paused contrary to FR-4	Acknowledged
[L-2] <code>WalletManager::setSpecialWallet</code> guard is inert under the stub registry, letting an Issuer label any token-holding wallet as a platform wallet and bypass an active lockup	Resolved

7 Findings

7.1 Medium Risk

7.1.1 ComplianceServicePermissionless::_lockedAt and ComplianceServicePermissionless::_cleanupIssuances perform unguarded timestamp + lockPeriod addition, allowing a transfer agent to freeze transfers and issuances

Description: [PR-87](#) introduces ComplianceServicePermissionless with its own wallet-keyed lockup engine.

Both internal helpers compute timestamp + lockPeriod in checked arithmetic, with no upper-bound guard:

```
// contracts/compliance/ComplianceServicePermissionless.sol
function _lockedAt(address _wallet, uint256 _time) internal view returns (uint256) {
    uint256 lockPeriod = getComplianceConfigurationService().getNonUSLockPeriod();
    if (lockPeriod == 0) return 0;
    ...
    for (uint256 i = 0; i < count; i++) {
        if (walletIssuancesTimestamps[_wallet][i] + lockPeriod > _time) { // @audit unchecked addition
            ↪ overflow panic revert
            totalLockedShares += walletIssuancesValues[_wallet][i];
        }
    }
    ...
}

function _cleanupIssuances(address _wallet) internal {
    uint256 lockPeriod = getComplianceConfigurationService().getNonUSLockPeriod();
    ...
    while (currentIndex < currentCount) {
        if (walletIssuancesTimestamps[_wallet][currentIndex] + lockPeriod <= block.timestamp) { //
            ↪ @audit same issue
            ...
        }
    }
}
}
```

The same bug class was identified and fixed for ComplianceServiceRegulated:cleanupInvestorIssuances **inside this same PR** (commit 337f0ff feature@BC-2118: Fix cleanup_underflow_dos_on_large_lock_period):

```
// contracts/compliance/ComplianceServiceRegulated.sol - line 902
uint256 time = block.timestamp;

if (lockTime > time) return; // @audit fix added in this PR - NOT ported to permissionless

uint256 currentIssuancesCount = issuancesCounters[investor];
```

The same early-return guard was not applied to the two new permissionless helpers, even though they share the identical arithmetic pattern.

There are two distinct ways to trigger the overflow:

Variant A — `setNonUSLockPeriod(type(uint256).max)` ComplianceConfigurationService:setNonUSLockPeriod is onlyTransferAgentOrAbove. Setting it to `type(uint256).max` makes every record's timestamp + lockPeriod overflow on every call. Every transfer routes through ComplianceService:validateTransfer → ComplianceServicePermissionless:newPreTransferCheck → `_lockedAt` and panics.

Every issuance to any non-platform wallet with at least one prior record routes through ComplianceService:validateIssuance → ComplianceServicePermissionless:recordIssuance → `_cleanupIssuances` and panics. The token becomes fully frozen.

Variant B — Issuer-supplied `_issuanceTime = type(uint256).max` The permissionless override of `ComplianceServicePermissionless:validateIssuanceTime` returns `_issuanceTime` unchanged (it drops the `disallowBackDating` clamp from `ComplianceService:validateIssuanceTime`). An issuer call `dsToken.issueTokensCustom(victim, 1, type(uint256).max, 0, "", 0)` writes `walletIssuancesTimestamps[victim][count] = 2**256 - 1`. Any subsequent `_lockedAt(victim, _)` and `_cleanupIssuances(victim)` panic on that slot — permanently DoSing both transfers from and issuances to victim. Unlike Variant A this is per-wallet and cannot be cleared by reading `_cleanupIssuances` (which itself panics on the poisoned record).

Impact:

- *Variant A:* Token-wide DoS of all transfers and further issuances. Recovery requires the Transfer Agent to call `setNonUSLockPeriod(0)` or any value such that $(2^{256} - 1) - \max(\text{timestamps})$ does not overflow — feasible but destroys the lockup configuration in the process.
- *Variant B:* permanent per-wallet DoS for the targeted address. `_cleanupIssuances` cannot remove the poisoned record because it itself panics evaluating the slot. The only recovery is `setNonUSLockPeriod(0)` token-wide, which abandons the lockup mechanism — the only on-chain compliance enforcement in the permissionless model.

Proof of Concept: Run the following test:

```
import hre from "hardhat";
import { expect } from "chai";
import { loadFixture, time } from "@nomicfoundation/hardhat-toolbox/network-helpers";
import { deployDSTokenPermissionless, DAYS } from "../utils/fixture";
import { DSConstants } from "../utils/globals";

describe(" setNonUSLockPeriod(type(uint256).max) freezes transfers and issuance", function () {
  const LOCK_PERIOD = 30 * DAYS;

  async function fixtureWithLockupAndIssuance() {
    const contracts = await loadFixture(deployDSTokenPermissionless);
    const { dsToken, trustService, complianceService, complianceConfigurationService } = contracts;

    const [master, transferAgent, user1, user2] = await hre.ethers.getSigners();
    await trustService.connect(master).setRole(transferAgent, DSConstants.roles.TRANSFER_AGENT);

    const user1Address = await user1.getAddress();
    const user2Address = await user2.getAddress();

    await complianceConfigurationService.connect(transferAgent).setNonUSLockPeriod(LOCK_PERIOD);
    await dsToken.issueTokens(user1Address, 1_000);

    return { dsToken, complianceService, complianceConfigurationService, transferAgent, user1,
      ↪ user1Address, user2, user2Address };
  }

  it("setNonUSLockPeriod(type(uint256).max) panics every transfer-path call", async function () {
    const { dsToken, complianceService, complianceConfigurationService, transferAgent, user1,
      ↪ user1Address, user2Address } = await fixtureWithLockupAndIssuance();

    // ATTACK: Transfer Agent sets lockPeriod to type(uint256).max
    await
      ↪ complianceConfigurationService.connect(transferAgent).setNonUSLockPeriod(hre.ethers.MaxUint256);

    // Every transfer-path call now hits `timestamps[i] + lockPeriod` overflow.
    await expect(complianceService.lockedAt(user1Address, await
      ↪ time.latest()).to.be.revertedWithPanic(0x11);
    await expect(complianceService.preTransferCheck(user1Address, user2Address,
      ↪ 1)).to.be.revertedWithPanic(0x11);
    await expect(dsToken.connect(user1).transfer(user2Address, 1)).to.be.revertedWithPanic(0x11);
  });
});
```

```

    // Issuance to a wallet with an existing record routes through _cleanupIssuances + panics too.
    await expect(dsToken.issueTokens(user1Address, 1)).to.be.revertedWithPanic(0x11);
  });

  it("token-wide freeze - every holder is affected", async function () {
    const { dsToken, complianceConfigurationService, transferAgent, user1, user1Address, user2Address }
      = await fixtureWithLockupAndIssuance();
    await dsToken.issueTokens(user2Address, 500);

    await
      complianceConfigurationService.connect(transferAgent).setNonUSLockPeriod(hre.ethers.MaxUint256);

    await expect(dsToken.connect(user1).transfer(user2Address, 1)).to.be.revertedWithPanic(0x11);
    const user2 = (await hre.ethers.getSigners())[3];
    await expect(dsToken.connect(user2).transfer(user1Address, 1)).to.be.revertedWithPanic(0x11);
    await expect(dsToken.issueTokens(user1Address, 1)).to.be.revertedWithPanic(0x11);
    await expect(dsToken.issueTokens(user2Address, 1)).to.be.revertedWithPanic(0x11);
  });

  it("recovery requires setNonUSLockPeriod(0), destroying the lockup configuration", async function () {
    const { dsToken, complianceConfigurationService, transferAgent, user1, user1Address, user2Address }
      = await fixtureWithLockupAndIssuance();

    await
      complianceConfigurationService.connect(transferAgent).setNonUSLockPeriod(hre.ethers.MaxUint256);
    await expect(dsToken.connect(user1).transfer(user2Address, 1)).to.be.revertedWithPanic(0x11);

    // Only escape: abandon the lockup mechanism entirely.
    await complianceConfigurationService.connect(transferAgent).setNonUSLockPeriod(0);
    await expect(dsToken.connect(user1).transfer(user2Address, 1)).to.not.be.reverted;
    expect(await dsToken.balanceOf(user2Address)).to.equal(1);
  });
});

```

Recommended Mitigation: Consider implementing following fix:

```

function _lockedAt(address _wallet, uint256 _time) internal view returns (uint256) {
    uint256 lockPeriod = getComplianceConfigurationService().getNonUSLockPeriod();
    if (lockPeriod == 0) return 0;
+   if (lockPeriod > _time) return 0;

    uint256 count = walletIssuancesCounters[_wallet];
    if (count == 0) return 0;
    ...
}

function _cleanupIssuances(address _wallet) internal {
    uint256 lockPeriod = getComplianceConfigurationService().getNonUSLockPeriod();
+   if (lockPeriod > block.timestamp) return;
    uint256 currentCount = walletIssuancesCounters[_wallet];
    ...
}

function validateIssuanceTime(uint256 _issuanceTime)
    public view virtual override returns (uint256)
{
-   return _issuanceTime;
+   return _issuanceTime > block.timestamp ? block.timestamp : _issuanceTime;
}

```

Securitize: Fixed in [a36bb0a](#).

Cyfrin: Verified.

7.1.2 TokenLibrary::issueTokensCustom routes every manual lock under permissionless into a single shared investorsLocks[""] bucket, exhausting the global 30-lock cap and DoS-ing all future issuances with manual locks

Description: PR-87's design premise is that the `!CommonUtils.isEmptyString(investor)` guard at DSToken:296 and TokenLibrary:160 makes every investor-keyed bookkeeping path a no-op under StubRegistryService (whose `getInvestor(addr)` always returns "").

The audit-scope document (FR-1) makes this explicit: *"tokenData.investorsBalances must never be written under this configuration."*

That guard exists for `investorsBalances`. It does **not** exist for `InvestorLockManager.investorsLocks`. The manual-lock write path is unconditional:

```
// token/TokenLibrary.sol - issueTokensCustom
function issueTokensCustom(...) public returns (uint256) {
    ...
    IDSComplianceService(_services[COMPLIANCE_SERVICE]).validateIssuance(_params._to, _params._value,
    ↪ _params._issuanceTime);
    ...
    for (uint256 i = 0; i < _params._valuesLocked.length; i++) {
        totalLocked += _params._valuesLocked[i];
        _lockManager.addManualLockRecord(_params._to, _params._valuesLocked[i], _params._reason,
        ↪ _params._releaseTimes[i]); // @audit no dead-branch guard here
    }
    ...
}
```

```
// compliance/InvestorLockManager.sol
function addManualLockRecord(address _to, uint256 _valueLocked, string calldata _reason, uint256
    ↪ _releaseTime) public override onlyTransferAgentOrAboveOrToken {
    require(_to != address(0), "Invalid address");
    createLock(_to, _valueLocked, 0, _reason, _releaseTime);
}

function createLock(address _to, uint256 _valueLocked, ...) internal {
    createLockForInvestor(getRegistryService().getInvestor(_to), _valueLocked, _reasonCode,
    ↪ _reasonString, _releaseTime); // @audit getInvestor(_to) returns "" under stub every wallet's
    ↪ lock lands under the SAME key
    ...
}

function createLockForInvestor(string memory _investor, uint256 _valueLocked, ...) public override
    ↪ validLock(...) onlyTransferAgentOrAboveOrToken {
    uint256 totalLockCount = investorsLocksCounts[_investor];
    require(totalLockCount < MAX_LOCKS_PER_INVESTOR, "Too many locks for this investor"); // @audit
    ↪ MAX_LOCKS_PER_INVESTOR = 30, applied to investorId="" globally under stub
    setLockInfoImpl(_investor, totalLockCount, _valueLocked, _reasonCode, _reasonString, _releaseTime);
    ...
}
```

Compare this with the analogous guard that the diff DOES enforce on `investorsBalances`:

```
// token/TokenLibrary.sol - updateInvestorBalance
function updateInvestorBalance(TokenData storage _tokenData, IDRegistryService _registryService,
    ↪ address _wallet, uint256 _shares, CommonUtils.IncDec _increase) internal {
    string memory investor = _registryService.getInvestor(_wallet);
    if (!CommonUtils.isEmptyString(investor)) { // @audit dead-branch guard present here
        ...
        _tokenData.investorsBalances[investor] = balance;
    }
}
```



```

    }
}

```

Under the stub:

1. `DSToken.issueTokensCustom(to, value, t, _valueLocked > 0, reason, releaseTime)` (or `issueTokensWithMultipleLocks` with a non-empty `_valuesLocked`, or `TokenIssuer.issueTokens` with non-empty `_locksValues`, or `BulkOperator.bulkRegisterAndIssuance` with non-empty entry `locksValues`) — all onlyIssuerOrAbove.
2. Reaches `TokenLibrary.issueTokensCustom`'s loop at line 93-96, calls `addManualLockRecord` for each entry.
3. `createLock(_to, ...)` calls `getRegistryService().getInvestor(_to)` — the stub returns `""`.
4. `createLockForInvestor("", _valueLocked, ...)` writes to `investorsLocks[""][count]` and increments `investorsLocksCounts[""]`.
5. After 30 such writes — across ANY combination of wallets — `require(totalLockCount < MAX_LOCKS_PER_INVESTOR)` at `InvestorLockManager:52` reverts, propagating up through the loop and reverting the entire `issueTokens*` transaction.

Impact:

- After 30 cumulative issuances-with-locks across the entire token, `Issuer` is locked out of the entire issuance-with-locks workflow. The DoS is silent: the operator sees "Too many locks for this investor" — a confusing error mentioning an investor concept that the permissionless model is supposed to have eliminated. Recovery requires the Transfer Agent to manually call `InvestorLockManager.removeLockRecordForInvestor("", index)` repeatedly to free slots.
- `InvestorLockManager.lockCount(addr)`, `lockInfo(addr, idx)`, and `getTransferableTokens(addr, t)` all key off `getInvestor(addr)`. Under the stub they all return data from `investorsLocks[""]`. Any caller — operator dashboard, off-chain reconciliation, future contract integration — that asks "how many locks does wallet X have?" gets the GLOBAL count regardless of X.

Proof of Concept: Run the following test:

```

import hre from "hardhat";
import { expect } from "chai";
import { loadFixture, time } from "@nomicfoundation/hardhat-toolbox/network-helpers";
import { deployDSTokenPermissionless, DAYS } from "../utils/fixture";

describe("PoC M-01 Issuance with locks pollutes shared investorsLocks[\"\"] bucket and exhausts global
↳ cap", function () {
  async function permissionlessFixture() {
    const contracts = await loadFixture(deployDSTokenPermissionless);
    const { dsToken, lockManager } = contracts;
    const [master, user1, user2, user3] = await hre.ethers.getSigners();
    return {
      dsToken, lockManager, master, user1, user2, user3,
      user1Address: await user1.getAddress(),
      user2Address: await user2.getAddress(),
      user3Address: await user3.getAddress(),
    };
  };
}

it("PoC: a single issuance with `_valueLocked > 0` writes to investorsLocks[\"\"]", async function ()
↳ {
  const { dsToken, lockManager, user1Address } = await permissionlessFixture();
  expect(await lockManager.lockCountForInvestor("")).to.equal(0);
  const releaseTime = (await time.latest()) + 30 * DAYS;
  await dsToken.issueTokensCustom(user1Address, 50, await time.latest(), 1, "lock-1", releaseTime);
  expect(await lockManager.lockCountForInvestor("")).to.equal(1);
});

```

```

it("PoC: locks from DIFFERENT wallets all collapse into the same investorsLocks[\"\"] bucket", async
  ↪ function () {
    const { dsToken, lockManager, user1Address, user2Address, user3Address } = await
      ↪ permissionlessFixture();
    const releaseTime = (await time.latest()) + 30 * DAYS;
    await dsToken.issueTokensCustom(user1Address, 50, await time.latest(), 1, "lock-u1", releaseTime);
    await dsToken.issueTokensCustom(user2Address, 50, await time.latest(), 2, "lock-u2", releaseTime);
    await dsToken.issueTokensCustom(user3Address, 50, await time.latest(), 3, "lock-u3", releaseTime);

    expect(await lockManager.lockCountForInvestor("")).to.equal(3);

    // Cross-wallet leakage: per-address queries all return the SAME shared value
    expect(await lockManager.lockCount(user1Address)).to.equal(3);
    expect(await lockManager.lockCount(user2Address)).to.equal(3);
    expect(await lockManager.lockCount(user3Address)).to.equal(3);
  });

it("PoC: 30 issuances-with-locks exhaust the shared cap, DoS-ing further issuances globally", async
  ↪ function () {
    const { dsToken, lockManager } = await permissionlessFixture();
    const signers = await hre.ethers.getSigners();
    const releaseTime = (await time.latest()) + 30 * DAYS;

    // 30 issuances, round-robin across wallets - no individual wallet receives many locks.
    for (let i = 0; i < 30; i++) {
      const recipient = await signers[(i % 10) + 1].getAddress();
      await dsToken.issueTokensCustom(recipient, 10, await time.latest(), 1, `lock-${i}`, releaseTime);
    }
    expect(await lockManager.lockCountForInvestor("")).to.equal(30);

    // 31st issuance to a fresh wallet reverts at addManualLockRecord even though that
    // wallet has zero individual locks. The cap is global because the bucket is shared.
    const freshRecipient = await signers[15].getAddress();
    await expect(
      dsToken.issueTokensCustom(freshRecipient, 10, await time.latest(), 1, "lock-31", releaseTime)
    ).to.be.revertedWith("Too many locks for this investor");

    // Issuance without a manual lock continues to work - only the locked-issuance workflow is DoS-ed.
    await expect(dsToken.issueTokens(freshRecipient, 10)).to.not.be.reverted;
  });
});

```

Recommended Mitigation: Consider adding following fix:

```

// compliance/InvestorLockManager.sol
function createLockForInvestor(string memory _investor, uint256 _valueLocked, uint256 _reasonCode,
  ↪ string calldata _reasonString, uint256 _releaseTime)
  public
  override
  validLock(_valueLocked, _releaseTime)
  onlyTransferAgentOrAboveOrToken
{
  + require(!CommonUtils.isEmptyString(_investor), "Empty investor ID");
  uint256 totalLockCount = investorsLocksCounts[_investor];
  require(totalLockCount < MAX_LOCKS_PER_INVESTOR, "Too many locks for this investor");
  setLockInfoImpl(_investor, totalLockCount, _valueLocked, _reasonCode, _reasonString, _releaseTime);
  ...
}

```

Securitize: Fixed in [4b689f6](#).

Cyfrin: Verified.

7.1.3 ComplianceServicePermissionless::recordIssuance writes a lockup record even when the lock period is zero, causing tokens minted with no lockup to be retroactively locked when a lockup is later enabled

Description: The audit-scope document FR-6 specifies: *"The lockup window is read from ComplianceConfigurationService.getNonUSLockPeriod() at evaluation time (not stored per-record). Period = 0 → no lockup enforced and no lockup record written."*

ComplianceServicePermissionless:recordIssuance does not honor the "no lockup record written" half of FR-6. It unconditionally writes an issuance record for every non-platform recipient, regardless of the configured lock period:

```
// compliance/ComplianceServicePermissionless.sol
function recordIssuance(address _to, uint256 _value, uint256 _issuanceTime)
    internal virtual override returns (bool)
{
    if (getWalletManager().isPlatformWallet(_to)) {
        return true; // @audit only platform wallets are exempt
    }

    _cleanupIssuances(_to);
    require(walletIssuancesCounters[_to] < MAX_ISSUANCES_PER_WALLET, "Issuance cap reached");

    uint256 shares = getRebasingProvider().convertTokensToShares(_value);
    uint256 count = walletIssuancesCounters[_to];
    walletIssuancesValues[_to][count] = shares; // @audit record written
    walletIssuancesTimestamps[_to][count] = _issuanceTime; // @audit even when getNonUSLockPeriod() == 0
    walletIssuancesCounters[_to] = count + 1;

    emit IssuanceRecorded(_to, shares, _issuanceTime);
    return true;
}
```

The danger is that ComplianceServicePermissionless:_lockedAt reads the lock period at *evaluation* time, not at *record* time:

```
// compliance/ComplianceServicePermissionless.sol
function _lockedAt(address _wallet, uint256 _time) internal view returns (uint256) {
    uint256 lockPeriod = getComplianceConfigurationService().getNonUSLockPeriod();
    if (lockPeriod == 0) return 0; // @audit dormant only while the period stays 0
    ...
    for (uint256 i = 0; i < count; i++) {
        if (walletIssuancesTimestamps[_wallet][i] + lockPeriod > _time) { // @audit old period-0 record
            ↪ re-evaluated against the NEW period
            totalLockedShares += walletIssuancesValues[_wallet][i];
        }
    }
    ...
}
```

A record written during a period-0 window is harmless *only while the period remains 0* (because _lockedAt short-circuits). The instant a non-zero lock period is configured, every such record whose original _issuanceTime is within lockPeriod seconds of "now" is treated as locked — retroactively freezing tokens that were minted as freely transferable.

Note that the period-0 case IS handled in _lockedAt (early return) and in _cleanupIssuances (records sweep when timestamp + 0 ≤ block.timestamp), but it is NOT handled in recordIssuance, which is exactly where FR-6 places the responsibility.

Consider following sequence:

```
1. Issuer mints 1000 to user1 while getNonUSLockPeriod() == 0 (the deployment default).
   ↪ walletIssuancesTimestamps[user1][0] = T1, walletIssuancesCounters[user1] = 1
```

- transfers work, because `_lockedAt` short-circuits on `lockPeriod == 0`
- 2. `user1` treats the 1000 tokens as freely transferable (they were minted with no lockup).
- 3. Transfer Agent later enables a 30-day lockup for go-forward issuances:
`setNonUSLockPeriod(30 days)`
- 4. `user1` attempts to transfer → `_lockedAt(user1, now)` now sees `lockPeriod = 30d`,
evaluates `T1 + 30d > now` → `true` → 1000 locked → transfer reverts with code 16.
- 5. The lock persists until `T1 + 30 days` - anchored to the ORIGINAL mint time, not to the policy-change time.

Impact: When a Transfer Agent enables a lockup, every holder who received a period-0 mint within the trailing `lockPeriod` window is simultaneously and unexpectedly frozen — a transfer DoS for up to `lockPeriod` from each mint's original timestamp.

It is noted that there is no permanent loss (the lock expires at `mintTime + lockPeriod`), but holders who acquired or planned around the freely-transferable status are blocked for a bounded but potentially long window.

Proof of Concept: Run the following test:

```
import hre from "hardhat";
import { expect } from "chai";
import { loadFixture, time } from "@nomicfoundation/hardhat-toolbox/network-helpers";
import { deployDSTokenPermissionless, DAYS } from "../utils/fixture";
import { DSConstants } from "../utils/globals";

describe("PoC M-02 period-0 mint creates a record that a later lockup retroactively locks", function () {
  ↪ () {
    async function fixture() {
      const contracts = await loadFixture(deployDSTokenPermissionless);
      const { dsToken, trustService, complianceService, complianceConfigurationService } = contracts;
      const [master, transferAgent, user1, user2] = await hre.ethers.getSigners();
      await trustService.connect(master).setRole(transferAgent, DSConstants.roles.TRANSFER_AGENT);
      return {
        dsToken,
        complianceService,
        complianceConfigurationService,
        transferAgent,
        user1,
        user2,
        user1Address: await user1.getAddress(),
        user2Address: await user2.getAddress(),
      };
    }

    it("PoC: mint at lockPeriod==0 records an issuance despite FR-6", async function () {
      ↪ const { dsToken, complianceService, complianceConfigurationService, user1Address } = await
        fixture();

      expect(await complianceConfigurationService.getNonUSLockPeriod()).to.equal(0);

      await dsToken.issueTokens(user1Address, 1_000);

      // FR-6 says NO record should be written at period 0 - but one is.
      expect(await complianceService.issuancesCount(user1Address)).to.equal(1);

      // At period 0 the token is still transferable (lockedAt short-circuits on lockPeriod==0)
      const now = await time.latest();
      expect(await complianceService.lockedAt(user1Address, now + 1)).to.equal(0);
    });

    it("PoC: enabling a lockup later retroactively locks the period-0 mint", async function () {
      ↪ const { dsToken, complianceService, complianceConfigurationService, transferAgent, user1,
        user1Address, user2Address } = await fixture();
```

```

// 1. Mint 1000 while there is NO lockup. Holder reasonably expects free transfer forever.
await dsToken.issueTokens(user1Address, 1_000);

let check = await complianceService.preTransferCheck(user1Address, user2Address, 1_000);
expect(check[0]).to.equal(0n); // VALID

// 2. One day later the Transfer Agent enables a 30-day lockup (for go-forward issuances).
await time.increase(1 * DAYS);
await complianceConfigurationService.connect(transferAgent).setNonUSLockPeriod(30 * DAYS);

// 3. The OLD mint is now retroactively locked, even though it predates the lockup.
check = await complianceService.preTransferCheck(user1Address, user2Address, 1_000);
expect(check[0]).to.equal(16n); // TOKENS_LOCKED
expect(check[1]).to.equal("Tokens Locked");
await expect(dsToken.connect(user1).transfer(user2Address, 1_000)).to.be.reverted;

// lockedAt reports the full balance as locked, anchored to the ORIGINAL mint time
const now = await time.latest();
expect(await complianceService.lockedAt(user1Address, now)).to.equal(1_000);

// 4. The lock persists until 30 days after the ORIGINAL issuance, not the policy change.
await time.increase(29 * DAYS);
check = await complianceService.preTransferCheck(user1Address, user2Address, 1_000);
expect(check[0]).to.equal(0n); // VALID again once original-mint + 30d elapses
});
});

```

Recommended Mitigation: Consider following fix:

```

function recordIssuance(address _to, uint256 _value, uint256 _issuanceTime)
    internal virtual override returns (bool)
{
    if (getWalletManager().isPlatformWallet(_to)) {
        return true;
    }

+   if (getComplianceConfigurationService().getNonUSLockPeriod() == 0) {
+       return true;
+   }

    _cleanupIssuances(_to);

    ...
}

```

Securitize: Fixed in [f6f44f6](#).

Cyfrin: Verified.

7.2 Low Risk

7.2.1 ComplianceServicePermissionless::preIssuanceCheck omits the pause check, allowing issuance while the token is paused contrary to FR-4

Description: The audit-scope document FR-4 states: "All transfers and issuances must be rejected with code 10 while paused. Burn and seize are exempt from the pause check."

Transfers honor this. Issuance does not — the pause flag is never read on the issuance path.

The new permissionless preIssuanceCheck checks only zero-address and blacklist:

```
// compliance/ComplianceServicePermissionless.sol
function preIssuanceCheck(address _to, uint256 /*_value*/)
    public view virtual override returns (uint256 code, string memory reason)
{
    if (_to == address(0)) {
        return (101, "Zero address");
    }
    if (getBlackListManager().isBlacklisted(_to)) {
        return (100, WALLET_BLACKLISTED); // @audit no code-10 pause branch FR-4 requires rejecting
        ↪ issuance while paused
    }
    return (0, VALID);
}
```

The enforcing function ComplianceService:validateIssuance likewise never reads the pause flag:

```
// compliance/ComplianceService.sol
function validateIssuance(address _to, uint256 _value, uint256 _issuanceTime) public override onlyToken
    ↪ returns (bool) {
    ...
    require(authorizedSecurities == 0 || totalSupply + _value <= authorizedSecurities,
        ↪ MAX_AUTHORIZED_SECURITIES_EXCEEDED);
    (code, reason) = preIssuanceCheck(_to, _value); // @audit no pause check inside; no whenNotPaused on
    ↪ caller
    require(code == 0, reason);
    uint256 issuanceTime = validateIssuanceTime(_issuanceTime);
    return recordIssuance(_to, _value, issuanceTime);
}
```

Contrast with the transfer path, which DOES honor the pause: DSToken:canTransfer passes paused into validateTransfer, and ComplianceServicePermissionless:newPreTransferCheck returns (10, TOKEN_PAUSED) as its first check:

```
// compliance/ComplianceServicePermissionless.sol - newPreTransferCheck
if (_pausedToken) {
    return (10, TOKEN_PAUSED); // @audit transfers correctly reject while paused issuance has no
    ↪ equivalent
}
```

Impact: An Issuer (or Master) can mint new tokens while the token is paused, contrary to FR-4. Pausing is an emergency control; allowing minting during that window weakens the "everything is frozen" guarantee operators expect, and permits supply inflation (holder dilution) during a freeze.

Recommended Mitigation: If FR-4 is the intended behavior, consider adding a pause branch to the permissionless preIssuanceCheck.

Securitize: Acknowledged. Working as designed, the spec document (FR-4) will be updated.

7.2.2 WalletManager::setSpecialWallet guard is inert under the stub registry, letting an Issuer label any token-holding wallet as a platform wallet and bypass an active lockup

Description: WalletManager::setSpecialWallet carries a defense intended to stop a wallet that belongs to a registered investor from being labeled a "special" (platform / issuer / exchange) wallet — special wallets receive compliance exemptions, so labeling an investor's wallet would let it escape those rules:

```
// compliance/WalletManager.sol
function setSpecialWallet(address _wallet, uint8 _type) internal override returns (bool) {
    require(CommonUtils.isEmptyString(getRegistryService().getInvestor(_wallet)), "Wallet belongs to
    ↳ investor"); // @audit always passes under the stub
    ...
    walletsTypes[_wallet] = _type;
    ...
}

function addPlatformWallet(address _wallet) public override onlyIssuerOrAbove returns (bool) {
    return setSpecialWallet(_wallet, PLATFORM);
}
```

Under StubRegistryService, getInvestor(addr) returns "" for every address, so isEmptyString(...) is always true and the require never blocks. The guard is structurally inert in the permissionless configuration.

Platform wallets are exempt from the lockup in ComplianceServicePermissionless::newPreTransferCheck:

```
// compliance/ComplianceServicePermissionless.sol
if (!getWalletManager().isPlatformWallet(_from)) { // @audit platform wallets skip the lockup
    ↳ check
    uint256 locked = _lockedAt(_from, block.timestamp);
    if (locked > 0 && _value > _balanceFrom - locked) {
        return (16, TOKENS_LOCKED);
    }
}
```

Consequently an onlyIssuerOrAbove caller can take any wallet that currently holds locked tokens, call addPlatformWallet(thatWallet) — which always succeeds under the stub — and the wallet's lockup is immediately bypassed: transfers from it stop returning code 16.

Impact: An Issuer can exempt any wallet from the lockup by labeling it a platform wallet, defeating the lockup for that address. The lockup is itself an Issuer/Transfer-Agent-configurable compliance feature, and no funds are lost — only the lockup timing guarantee for the affected wallet is removed.

Recommended Mitigation: Consider reverting the labeling of a wallet as PLATFORM while it has an active lockup.

Securitize: Fixed in [Oede206](#).

Cyfrin: Verified. Labeling a wallet as platform no longer bypasses active lockup records, since transfer-time lock checks now apply to platform wallets as well.